

LAPORAN TUGAS AKHIR

Mata Kuliah: Teknik Kompilasi



“Simulasi Proses Kompilasi Konstruksi While Loop Menggunakan Python”

Disusun oleh:

Zam Zam Okupa

NIM. 231011403009

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PAMULANG**

2025

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan Tugas Akhir mata kuliah Teknik Kompilasi ini dapat diselesaikan dengan baik. Laporan ini membahas simulasi proses kompilasi pada konstruksi perulangan *while loop* menggunakan bahasa pemrograman Python, mencakup tahapan analisis leksikal, sintaksis, semantik, hingga generasi *Three-Address Code*.

Penulis mengucapkan terima kasih yang sebesar-besarnya kepada Dosen Pengampu mata kuliah Teknik Kompilasi atas bimbingan dan ilmu yang telah diberikan selama perkuliahan. Ucapan terima kasih juga disampaikan kepada seluruh pihak yang telah membantu dalam penyelesaian tugas ini.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini bermanfaat bagi penulis khususnya dan pembaca pada umumnya.

Pamulang, Juni 2025

Penulis,

Zam Zam Okupa

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR TABEL	4
1 PENDAHULUAN	5
1.1 Latar Belakang	5
1.2 Rumusan Masalah	5
1.3 Tujuan	5
1.4 Manfaat	6
1.5 Batasan Implementasi	6
2 LANDASAN TEORI	7
2.1 Teknik Kompilasi	7
2.2 Analisis Leksikal	7
2.3 Analisis Sintaksis dan AST	7
2.4 Analisis Semantik dan Three-Address Code	7
3 ANALISIS & PERANCANGAN	8
3.1 Pola Tata Bahasa (BNF)	8
3.2 Rancangan Struktur Data (Token dan AST)	8
3.3 Alur Proses Kompilasi	8
4 IMPLEMENTASI & PENGUJIAN	9
4.1 Lingkungan Implementasi	9
4.2 Implementasi Kode (Bagian Penting)	9
4.3 Output Program	10
4.4 Pengujian	11
5 PENUTUP	12
5.1 Kesimpulan	12
5.2 Saran	12
DAFTAR PUSTAKA	13

LAMPIRAN	14
---------------------------	-----------

DAFTAR TABEL

3.1	Jenis Token yang Dikenali Lexer	8
4.1	Hasil Pengujian Fungsi Program	11

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Kompilasi merupakan proses penerjemahan kode sumber yang ditulis dalam bahasa pemrograman tingkat tinggi menjadi representasi yang dapat dipahami dan dieksekusi oleh mesin. Proses ini terdiri dari beberapa tahapan yang saling berkaitan, yaitu analisis leksikal, analisis sintaksis, analisis semantik, dan generasi kode antara. Pemahaman yang mendalam terhadap setiap tahapan ini sangat penting bagi seorang pengembang perangkat lunak.

Konstruksi perulangan *while loop* merupakan salah satu konstruksi fundamental dalam hampir semua bahasa pemrograman. Konstruksi ini memiliki struktur yang cukup representatif untuk menggambarkan bagaimana sebuah kompiler bekerja, mulai dari pengenalan token hingga pembentukan kode antara yang mengandung mekanisme percabangan dan lompatan (*branching*). Oleh karena itu, tugas akhir ini memilih konstruksi *while loop* sebagai objek simulasi proses kompilasi.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam tugas akhir ini adalah:

1. Bagaimana mendefinisikan pola tata bahasa (BNF) yang valid untuk konstruksi *while loop*?
2. Bagaimana mengimplementasikan tahapan analisis leksikal, sintaksis, dan semantik pada konstruksi *while loop*?
3. Bagaimana menghasilkan Three-Address Code (TAC) dari Abstract Syntax Tree (AST) yang terbentuk?

1.3 Tujuan

Tujuan dari tugas akhir ini adalah:

1. Mendefinisikan aturan tata bahasa BNF untuk konstruksi *while loop*.
2. Mengimplementasikan program simulasi kompilasi menggunakan bahasa Python.
3. Menghasilkan Three-Address Code sebagai keluaran akhir dari proses simulasi kompilasi.

1.4 Manfaat

Bagi penulis, tugas ini memberikan pemahaman praktis tentang cara kerja kompiler secara nyata. Bagi pembaca, laporan ini dapat menjadi referensi dalam mempelajari teknik kompilasi, khususnya implementasi tahapan analisis leksikal, sintaksis, semantik, dan generasi kode untuk konstruksi perulangan. Selain itu, kode sumber yang dihasilkan bersifat modular dan dapat dikembangkan lebih lanjut untuk konstruksi bahasa pemrograman lainnya.

1.5 Batasan Implementasi

Batasan implementasi dalam tugas akhir ini meliputi:

1. Konstruksi yang disimulasikan terbatas pada *while loop*.
2. Pernyataan di dalam blok loop hanya mendukung deklarasi variabel bertipe `int/float/bool` dan penugasan (`assignment`).
3. Bahasa implementasi yang digunakan adalah Python 3 tanpa dependensi eksternal.

BAB 2

LANDASAN TEORI

2.1 Teknik Kompilasi

Kompiler adalah program yang menerjemahkan kode sumber dari bahasa pemrograman tingkat tinggi menjadi bahasa mesin atau kode antara. Proses kompilasi dibagi menjadi dua fase utama: analisis (*front-end*) dan sintesis (*back-end*). Fase analisis mencakup analisis leksikal, sintaksis, dan semantik, sedangkan fase sintesis mencakup optimasi dan generasi kode (Aho et al., 2006).

2.2 Analisis Leksikal

Analisis leksikal adalah tahap pertama kompilasi yang bertugas memecah aliran karakter masukan menjadi unit-unit bermakna yang disebut *token*. Komponen yang melakukan ini disebut *lexer* atau *scanner*. Setiap token memiliki tipe (mis. IDENT, NUMBER, KEYWORD) dan nilai (*lexeme*). Implementasi lexer umumnya memanfaatkan ekspresi reguler (Regex).

2.3 Analisis Sintaksis dan AST

Analisis sintaksis (*parsing*) memeriksa apakah urutan token sesuai dengan tata bahasa yang didefinisikan. Hasilnya adalah *Abstract Syntax Tree* (AST), sebuah pohon hierarkis yang merepresentasikan struktur program. Metode *Recursive Descent Parsing* adalah teknik top-down yang menggunakan satu fungsi rekursif untuk setiap aturan produksi tata bahasa.

2.4 Analisis Semantik dan Three-Address Code

Analisis semantik memeriksa makna program, termasuk kesesuaian tipe data dan keberadaan variabel dalam tabel simbol (*symbol table*). *Three-Address Code* (TAC) adalah representasi kode antara di mana setiap instruksi memiliki paling banyak tiga operand, dengan format umum: *result = operand1 op operand2*. TAC menggunakan variabel sementara (t1, t2, ...) dan label (L1, L2, ...) untuk mengelola alur kontrol.

BAB 3

ANALISIS & PERANCANGAN

3.1 Pola Tata Bahasa (BNF)

Aturan tata bahasa dalam notasi Backus-Naur Form (BNF) yang mendefinisikan konstruksi while loop:

```

<while_stmt> ::= "while" "(" <condition> ")" "{" <statements> "}"
<condition>  ::= <expr> <op_rel> <expr>
<op_rel>     ::= "<=" | ">=" | "==" | "!=" | "<" | ">"
<statements> ::= { <decl_stmt> | <assign_stmt> }
<decl_stmt>  ::= <type> <identifier> "=" <expr> ";"
<assign_stmt> ::= <identifier> "=" <expr> ";"
<expr>       ::= <term> { <op_arith> <term> }
<term>       ::= NUMBER | BOOL | identifier
<type>       ::= "int" | "float" | "bool"

```

3.2 Rancangan Struktur Data (Token dan AST)

Tabel 3.1: Jenis Token yang Dikenali Lexer

Tippe Token	Contoh	Pola Regex
WHILE	while	\bwhile\b
IDENT	i, sum, temp	[A-Za-z_][A-Za-z0-9_]*
NUMBER	10, 3.14	\d+(\.\d*)?
OP_REL	<=, >=, ==, !=	<= > = != < >
OP_ARITH	+, -, *, /	[+*/]
ASSIGN	=	=

3.3 Alur Proses Kompilasi

Program simulasi menerima kode sumber berupa teks, kemudian memprosesnya melalui empat tahap secara berurutan: (1) Lexer memecah teks menjadi token, (2) Parser membangun AST menggunakan metode Recursive Descent, (3) SemanticAnalyzer menelusuri AST untuk memvalidasi tabel simbol, dan (4) TACGenerator menelusuri AST kembali untuk menghasilkan instruksi Three-Address Code. Setiap tahap menghasilkan keluaran yang menjadi masukan bagi tahap berikutnya.

BAB 4

IMPLEMENTASI & PENGUJIAN

4.1 Lingkungan Implementasi

Implementasi dilakukan pada lingkungan berikut:

- Sistem Operasi: Windows 10 / Linux Ubuntu 22.04
- Bahasa Pemrograman: Python 3.10+
- Editor: Visual Studio Code
- Dependensi eksternal: Tidak ada (hanya modul bawaan Python: re, dataclasses, typing)

4.2 Implementasi Kode (Bagian Penting)

Kode sumber terdiri dari empat komponen utama. Berikut adalah bagian-bagian kunci dari implementasi:

a) Lexer - Fungsi `lexer()` dengan Regex

```
MASTER_RE = re.compile(  
    "|".join(f"(?P<{name}>{pattern})" for name, pattern in  
        TOKEN_SPEC)  
)
```

b) Parser - Metode `parse_while()` sebagai titik masuk

```
def parse_while(self) -> ASTNode:  
    self.consume("WHILE")  
    self.consume("LPAREN")  
    cond = self.parse_condition()  
    self.consume("RPAREN")  
    self.consume("LBRACE")  
    body = self.parse_statements()  
    self.consume("RBRACE")  
    return ASTNode("WhileStmt", children=[cond, body])
```

c) TACGenerator - Pola TAC untuk WhileStmt

```
label_start = self.new_label() # L1
label_end   = self.new_label() # L2
self.emit(f"{label_start}:")
cond_temp = self.generate(node.children[0])
self.emit(f"ifFalse {cond_temp} goto {label_end}")
self.generate(node.children[1]) # body
self.emit(f"goto {label_start}")
self.emit(f"{label_end}:")
```

4.3 Output Program

Dengan input:

```
while ( i <= 10 ) {
    int temp = i + 1 ;
    sum = sum + temp ;
    i = i + 1 ;
}
```

Program menghasilkan TAC sebagai berikut:

```
L1:
t1 = i <= 10
ifFalse t1 goto L2
t2 = i + 1
temp = t2
t3 = sum + temp
sum = t3
t4 = i + 1
i = t4
goto L1
L2:
```

4.4 Pengujian

Tabel 4.1: Hasil Pengujian Fungsi Program

No	Skenario Uji	Hasil Diharapkan	Status
1	Input while loop valid dengan kondisi $\leq$ dan tiga pernyataan di body	27 token dihasilkan, AST terbentuk, tidak ada error semantik, 11 baris TAC	Berhasil
2	Deklarasi variabel ganda di dalam blok (int temp dua kali)	Error semantik: "Variabel 'temp' sudah dideklarasikan"	Berhasil
3	Variabel belum dideklarasikan digunakan dalam assignment ($x = x + 1$)	Error semantik: "Variabel 'x' tidak dideklarasikan"	Berhasil
4	Input sintaksis tidak valid (kurung kurawal tidak lengkap)	SyntaxError: "Diharapkan RBRACE, tetapi input sudah habis"	Berhasil

BAB 5

PENUTUP

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa:

1. Program simulasi kompilasi berhasil dibangun menggunakan Python 3, mencakup seluruh tahapan dari analisis leksikal hingga generasi *Three-Address Code*.
2. Pola BNF berhasil diimplementasikan ke dalam metode Recursive Descent Parser yang menghasilkan AST terstruktur.
3. Analisis semantik berhasil mendeteksi dua jenis kesalahan: deklarasi variabel ganda dan penggunaan variabel yang belum dideklarasikan.
4. TAC yang dihasilkan merepresentasikan alur kontrol *while loop* dengan tepat menggunakan instruksi label, *ifFalse*, dan *goto*.

5.2 Saran

Untuk pengembangan lebih lanjut, program ini dapat diperluas dengan menambahkan dukungan konstruksi *for loop* dan *if-else*, pengecekan kesesuaian tipe data secara menyeluruh, dukungan loop bersarang (*nested loop*), serta tahap optimasi kode sebelum generasi TAC. Selain itu, antarmuka berbasis grafis (GUI) dapat ditambahkan untuk memudahkan visualisasi setiap tahapan kompilasi secara interaktif.

DAFTAR PUSTAKA

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.
2. Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., & Langendoen, K. (2012). *Modern Compiler Design* (2nd ed.). Springer.
3. Python Software Foundation. (2024). *re — Regular expression operations*. <https://docs.python.org/3/library/re.html>
4. Modul Perkuliahan Teknik Kompilasi. (2025). Universitas Pamulang.

LAMPIRAN

1. File Source Code: `while_compiler.py` (terlampir)
2. File Dokumentasi: `dokumentasi_while_compiler.md` (terlampir)
3. Contoh output lengkap hasil eksekusi program (lihat Bab IV subbab 4.3).